

Smart Tank Monitoring System

Anno Accademico 2025-2026

Autori:
Davide Mancini
Filippo Ricciotti
Martina Malagoli

11 settembre 2026

Indice

1	Introduzione	2
1.1	Obiettivo	2
1.2	Componenti utilizzati	3
1.2.1	TMS - ESP32	3
1.2.2	WCS - Arduino	4
2	Architettura del Software	5
2.1	TMS (ESP32)	5
2.2	WCS (Arduino)	6
2.3	CUS (Java)	7
2.4	DBS - Dashboard Subsystem	9
2.5	Protocolli di Comunicazione	10
2.5.1	TMS verso CUS (MQTT)	10
2.5.2	CUS e WCS (Seriale, 115200 baud)	10
2.5.3	CUS e DBS (HTTP)	10

Capitolo 1

Introduzione

1.1 Obiettivo

L'obiettivo del progetto è creare un'infrastruttura per monitorare e regolare automaticamente il livello dell'acqua piovana raccolta in una cisterna, aprendo o chiudendo un canale di scarico in base a quanto sta salendo il livello.

Il sistema è stato considerato nei suoi quattro sottosistemi:

- Il **TMS** (Tank Monitoring Subsystem) gira su un ESP32 e si occupa solo di leggere il livello dell'acqua tramite un sensore sonar, pubblicandolo poi via MQTT.
- Il **WCS** (Water Channel Subsystem), su Arduino UNO, muove fisicamente la valvola tramite un servomotore, sia quando riceve un comando dal CUS sia quando l'operatore la controlla a mano con un potenziometro.
- Il **CUS** (Control Unit Subsystem) è il cervello del sistema: un'applicazione Java che riceve i dati dal TMS, decide come deve comportarsi la valvola e tiene aggiornati sia il WCS che la dashboard.
- Il **DBS** (Dashboard Subsystem) è una semplice pagina web che mostra lo stato del sistema e permette di cambiare modalità o regolare la valvola da remoto.

1.2 Componenti utilizzati

1.2.1 TMS - ESP32

Componente	Modello	Pin	Interfaccia
Sensore sonar TRIG	HC-SR04	32	Output
Sensore sonar ECHO	HC-SR04	34	Input
LED verde	Standard	14	Output
LED rosso	Standard	25	Output

Tabella 1.1: Componenti hardware del TMS

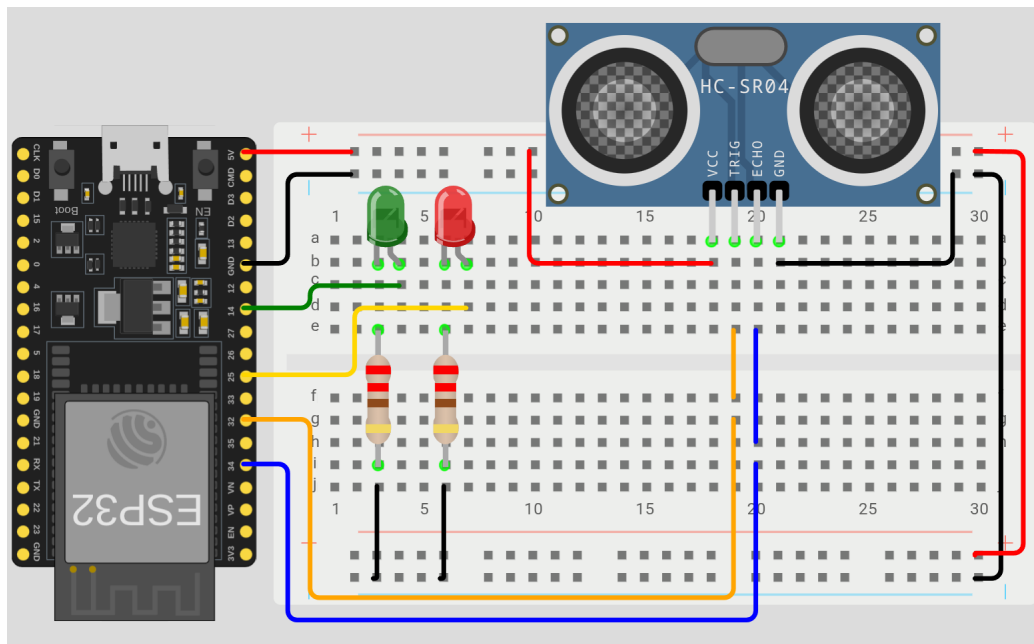


Figura 1.1: Schema di collegamento del TMS

1.2.2 WCS - Arduino

Componente	Modello	Pin	Interfaccia
Bottone	Standard	4	Input
Potenzimetro	Standard	A0	Analog
Servo motore	SG90	9	PWM
Display LCD I2C	16x2, indirizzo 0x27	A4/A5	I2C

Tabella 1.2: Componenti hardware del WCS

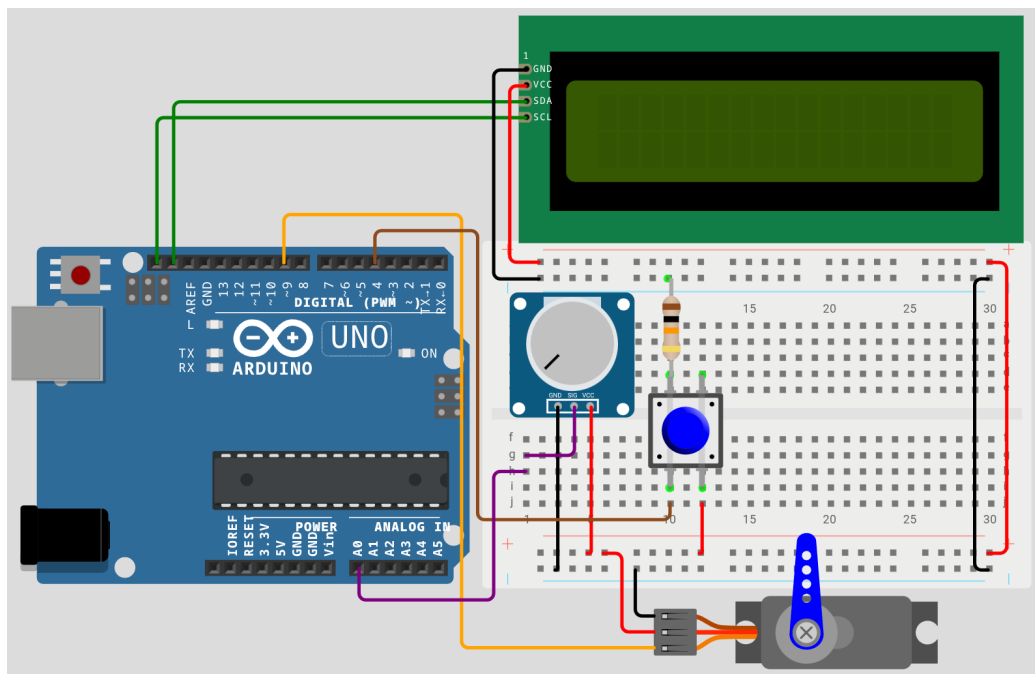


Figura 1.2: Schema di collegamento del WCS

Capitolo 2

Architettura del Software

2.1 TMS (ESP32)

Il firmware del TMS è organizzato in task su FreeRTOS, ciascuna con un compito isolato:

- **SonarTask** legge periodicamente il sensore e scrive il livello misurato nel `Context` condiviso.
- **MqttTask** tiene viva la connessione al broker, controlla lo stato della rete e pubblica un nuovo valore ogni volta che **SonarTask** ne produce uno.
- **LedTask** guarda semplicemente lo stato di rete riportato da **MqttTask** e decide quale dei due LED accendere: è di fatto una piccola macchina a stati a due soli stati, rete OK e problema di rete.

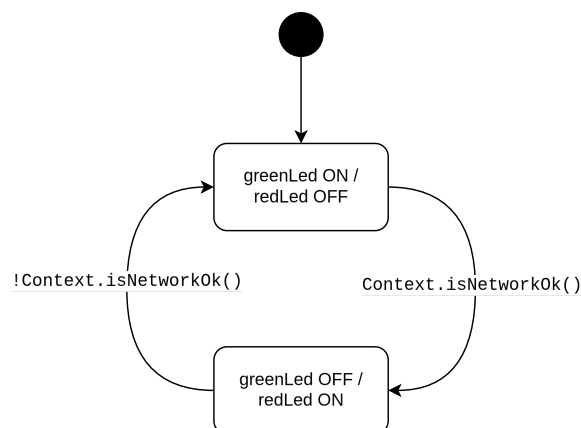


Figura 2.1: Macchina a stati di **LedTask**

2.2 WCS (Arduino)

Sul WCS è stato adottato un executive ciclico, basato su un timer hardware e uno scheduler che fa girare le task a periodi fissi:

- **SerialCommTask** legge e scrive i messaggi sulla seriale verso il CUS, secondo il protocollo descritto più avanti.
- **ModalityTask** gestisce il cambio tra **AUTOMATIC** e **MANUAL** quando viene premuto il pulsante, con debounce per evitare toggle multipli su una singola pressione.
- **PotentiometerTask** legge il potenziometro quando si è in **MANUAL**, con una deadband per evitare che sovrascriva il valore ricevuto da altri subsystem con il rumore del sensore.
- **ValveTask** è una macchina a stati con solo due stati (**IDLE** e **MOVING**) che sposta il servo verso il target a velocità costante, calcolando ad ogni tick lo spostamento in base al tempo realmente trascorso.
- **LCDTask** mostra sul display l'apertura corrente della valvola e la modalità in cui si trova il sistema, incluso lo stato **UNCONNECTED**.

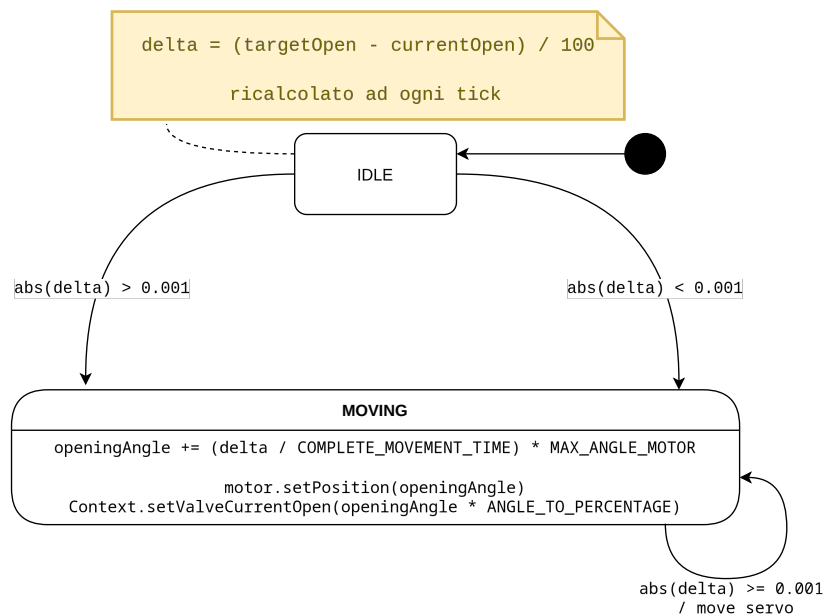


Figura 2.2: Macchina a stati di ValveTask

2.3 CUS (Java)

Il CUS è un'applicazione Java che si appoggia a Eclipse Paho per il client MQTT, a JSSC per la seriale e a Vert.x per il server HTTP verso il DBS. È composto da:

- **MqttListener**, che riceve i messaggi del livello dell'acqua dal TMS e li passa alla policy.
- **TankPolicy**, il componente che decide l'apertura della valvola quando in modalità **AUTOMATIC**:
una macchina a stati con quattro stati (**CLOSED**, **WAITING_T1**, **HALF_OPEN**, **FULL_OPEN**) che tiene conto delle soglie L1 e L2 e del tempo T1 richiesto prima di aprire parzialmente la valvola.
- **ConnectionWatchdog**, che tiene traccia di quanto tempo è passato dall'ultimo dato ricevuto dal TMS e fa scattare lo stato **UNCONNECTED** superato il tempo T2.
- **SerialService**, che parla con il WCS sullo stesso protocollo a righe descritto più avanti.
- **HttpApiServer**, che espone l'endpoint consumato dal DBS.
- **MainFrame**, un'interfaccia Swing che permette di vedere lo stato del sistema direttamente dal PC su cui gira il CUS.

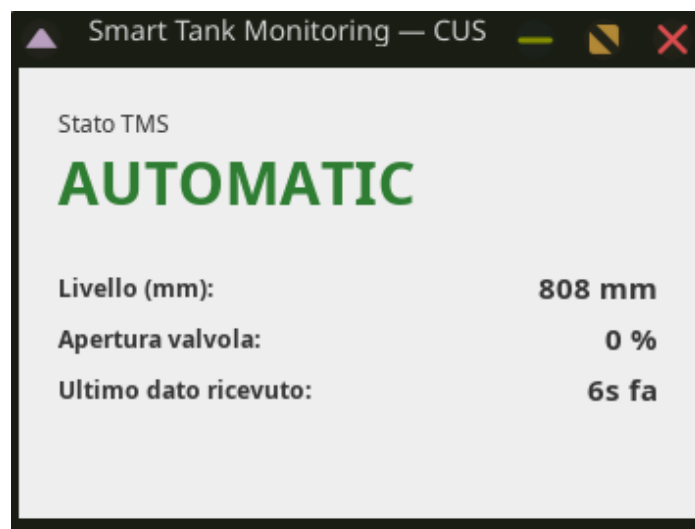


Figura 2.3: GUI del CUS

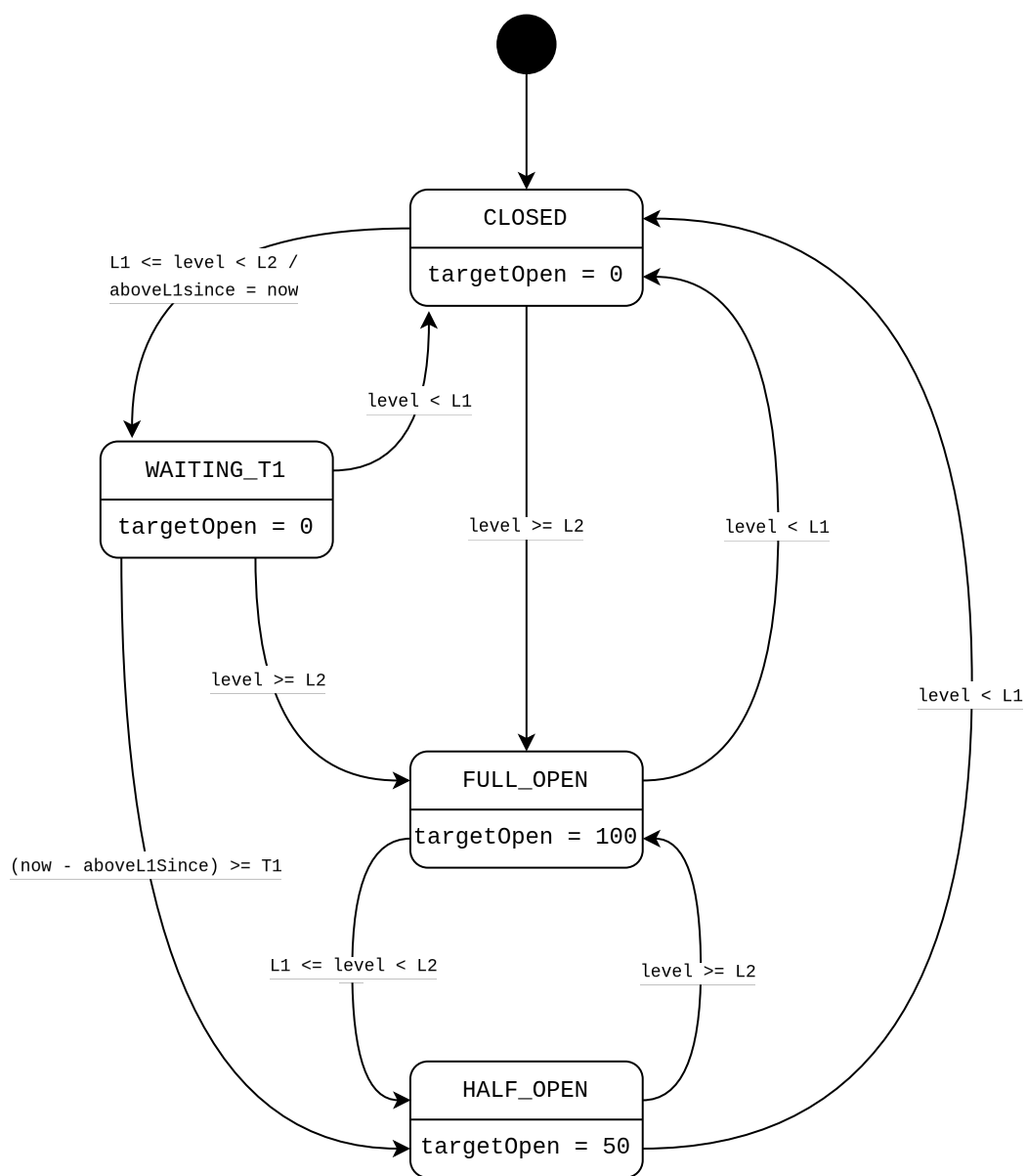


Figura 2.4: Macchina a stati di TankPolicy

2.4 DBS - Dashboard Subsystem

Il DBS è scritto in HTML, CSS e JavaScript puro, senza framework, con Chart.js per disegnare il grafico dello storico dei livelli. Comunica col CUS tramite `fetch()`: fa polling periodico su `GET /api/data` per restare aggiornato, e invia i comandi dell'operatore con `POST /api/data`.

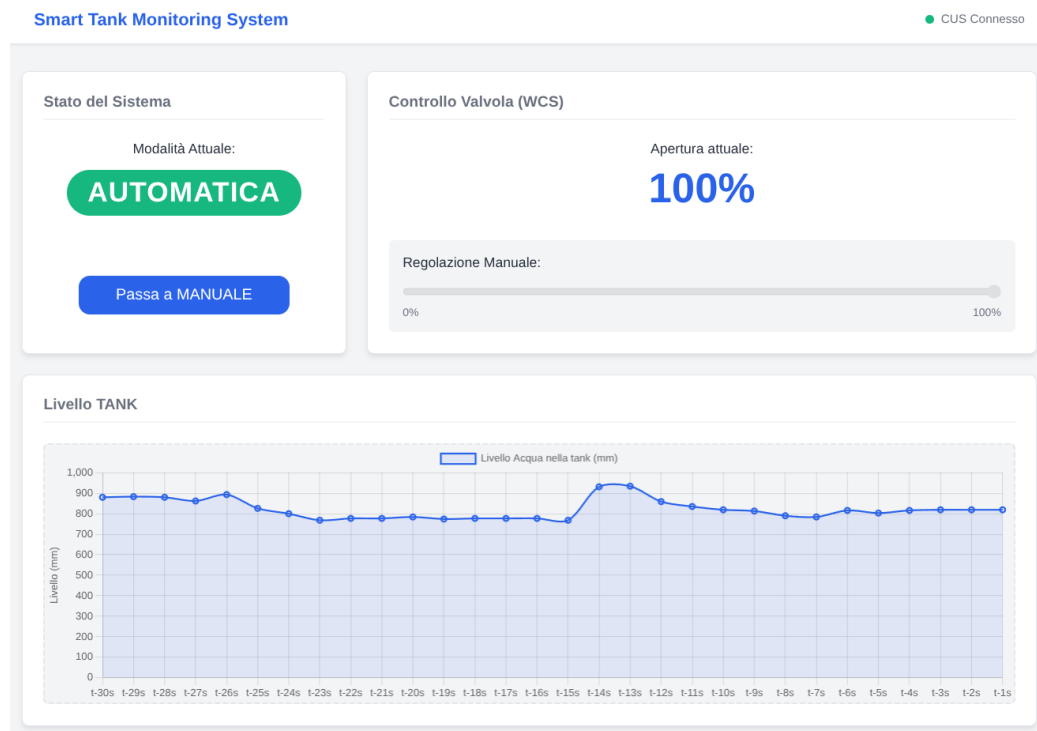


Figura 2.5: Sito web del DBS

2.5 Protocolli di Comunicazione

2.5.1 TMS verso CUS (MQTT)

Il TMS pubblica sul topic `tms/level` un messaggio testuale nel formato `level:#<valore>`, dove il valore rappresenta il livello misurato in millimetri. Il valore indica l'altezza in **mm** del livello dell'acqua in una cisterna da 1 metro.

2.5.2 CUS e WCS (Seriale, 115200 baud)

Lo scambio avviene tramite messaggi testuali terminati da `\n`, ciascuno formato da un prefisso seguito da due punti e da un valore.

Prefisso	Formato	Descrizione
<code>mode:</code>	<code>mode:AUTO</code> <code>mode:MANUAL</code>	Cambio di modalità, scambiato in entrambe le direzioni.
<code>open:</code>	<code>open:<0-100></code>	Apertura valvola, scambiato in entrambe le direzioni.
<code>net:</code>	<code>net:OK</code> <code>net:LOST</code>	Il CUS informa il WCS sullo stato del collegamento con il TMS.

Tabella 2.1: Messaggi scambiati tra CUS e WCS

2.5.3 CUS e DBS (HTTP)

Endpoint	Metodo	Descrizione
<code>/api/data</code>	GET	Restituisce lo stato corrente del sistema: modalità, apertura valvola e storico dei livelli.
<code>/api/data</code>	POST	Riceve modalità e/o apertura da applicare. Risponde con 409 se il sistema è in UNCONNECTED.

Tabella 2.2: API REST esposta dal CUS al DBS